Catchment Analyzer

Terrain-Driven Pond Siting and Catchment Estimation from Contour Maps

Phase 1 — Backend API Development

A contour map goes in; a pond site and its catchment come out.

Repository	https://github.com/devreddy1015/catchment_analyzer
API route	`POST http://10.1.75.53:5249/api/analyzeContour`
Alias route	`POST http://10.1.75.53:5249/api/findCatchment`
Live docs	http://10.1.75.53:5249/docs
Sample map	`contours_1m.kml` (1355 lines, 1 m interval)

Aman Pratap
Indian Institute of Technology Bhilai

September 1, 2026

Contents

1 Overview

This report documents a backend service that accepts a contour map as `KML` or `KMZ`, reconstructs the terrain surface it describes, routes water across that surface, and returns — as structured JSON — a recommended pond location together with the catchment that drains to it.

The central design commitment is that *nothing is specific to the sample map*. Extent, elevation range, contour interval, grid resolution and every score threshold are derived from whatever file is uploaded. The sample map is used to test the implementation, never to calibrate it. Section 4 sets out exactly how this is enforced.

> **What the service answers**
>
> Given only contour geometry, three questions are answered together: *where* should a pond go, *how much land* drains to it, and *how much water* that land yields. A fourth is answered implicitly — where a pond should *not* go, because the ground is a river, a floodplain, or a drainage line too large for a farm pond to hold.

1.1 Deliverables against the brief

Requirement	Where it is met
`POST /analyzeContour` or `/findCatchment`	Both exist; `/findCatchment` is an alias. §5
Accept contour map as uploaded file	`multipart/form-data`, KML and KMZ. §5
Analyze contour / terrain information	§3, stages 1–4
Identify a suitable pond location	§3.6
Estimate the catchment area	§3.7
Return structured JSON	§5.3
Demonstrate with the sample map	§6
No hard-coded locations or results	§4
Extensible to generalized maps	§7

Table 1: Requirement coverage.

2 System Architecture

The service is a FastAPI application. The HTTP layer is deliberately thin: it validates the upload and delegates immediately to a pure-Python analysis core that has no knowledge of HTTP. That separation is what makes the core reusable in Phase 2 from a queue worker, a batch script, or a notebook.

2.1 Processing pipeline

1. Parse KML/KMZ → contour lines + derived metadata

↓

2. Interpolate scattered contour vertices → regular elevation grid

↓

3. Condition priority-flood sink fill → depression depth + routable surface

↓

4. Route D8 flow direction → flow accumulation → stream network

↓

5. Classify river / floodplain / nala / still water → exclusion mask

↓

6. Score & rank four weighted suitability layers → candidate sites

↓

7. Delineate upstream walk from chosen outlet → catchment polygon

↓

8. Quantify storage, runoff yield, time of concentration → JSON + GeoJSON

2.2 Module map

Module	LOC	Responsibility
`core/kml.py`	295	Parse KML/KMZ; derive interval, bounds, levels
`core/grid.py`	183	Interpolate contours onto a square-cell grid
`core/hydrology.py`	408	Sink fill, D8 routing, accumulation, delineation
`core/siting.py`	518	Suitability scoring, storage, site ranking
`core/watercourse.py`	340	River / floodplain / nala classification
`core/pipeline.py`	533	Orchestration; assembles the response
`core/render.py`	100	Elevation, hillshade and watercourse overlays
`api/routes.py`	209	HTTP endpoints, validation, error mapping
`models.py`	271	Pydantic response schema
Total	**~3500**	

Table 2: Source layout. Each stage of the pipeline is one module with one job.

3 Catchment Estimation Approach

The method is the standard raster-hydrology chain — fill, route, accumulate, delineate — built on published algorithms rather than invented heuristics. What follows is the logic in pseudocode; the repository holds the implementation.

3.1 Stage 1 — Contour ingestion

A KMZ is a zip archive; the main `.kml` is extracted first. Elevation for each placemark is taken from whichever source the file actually provides: the coordinate z component, an `<ExtendedData>` field, or a numeric name such as `"272"`. Files in the wild use all three.

Algorithm 1 — Read contours from KML/KMZ

```
function read_contours(bytes, filename):
    if bytes is a zip archive then
        bytes <- extract largest .kml member   // KMZ case
    doc <- parse_xml(bytes)

    lines <- new list
    foreach placemark in doc:
        coords <- parse_coordinates(placemark)   // (lon, lat, z?) triples
        z <- first available of:
                 altitude from coordinates,
                 <ExtendedData> elevation field,
                 numeric <name> of the placemark
        if z is not null and coords has >= 2 points then
            append ContourLine(coords, z) to lines

    lines <- drop_isolated_levels(lines)   // spot heights, stray labels
    assert count(lines) >= 3 and distinct_levels(lines) >= 2

    // Everything below is DERIVED, never assumed:
    bounds   <- min/max of all vertex lon/lat
    levels   <- sorted distinct elevations
    interval <- median gap between neighbouring levels
    return ContourSet(lines, bounds, levels, interval)
```

`drop_isolated_levels` removes elevation values that do not belong to the contour series — a single placemark at 412 m among contours spaced every metre is a spot height, and interpolating it as terrain would raise a spurious hill. The test is relative to the series' own median spacing, so it holds for any interval.

3.2 Stage 2 — Surface reconstruction

Contours are scattered vertices carrying a z value. Turning them into a surface is a 2-D interpolation over a Delaunay triangulation, with nearest-neighbour fill for the region outside the convex hull, and a light uniform filter to suppress triangulation facets.

Algorithm 2 — Interpolate contours onto a grid

```
function build_grid(contours, resolution = 160, margin = 0.01):
    // Pad so edge contours are not clipped
    (s, w, n, e) <- contours.bounds expanded by margin

    // Cells are kept SQUARE IN METRES -- the D8 model assumes it.
    width_m  <- haversine(mid_lat, w, mid_lat, e)
    height_m <- haversine(s, w, n, w)
    if width_m >= height_m then
        cols <- resolution;  rows <- round(resolution * height_m / width_m)
    else
        rows <- resolution;  cols <- round(resolution * width_m / height_m)
    cell_size_m <- mean(width_m / cols, height_m / rows)

    // Control points: every vertex, decimated to a bounded budget
    step <- max(1, total_vertices / MAX_CONTROL_POINTS)
    (xy, z) <- vertices[::step] with their line elevations

    targets <- regular lon/lat mesh of shape (rows, cols)   // north-up
    values  <- LinearNDInterpolator(xy, z)(targets)         // Delaunay

```

```
    outside <- cells where values is NaN                    // beyond hull
    values[outside] <- NearestNDInterpolator(xy, z)(targets[outside])

    surface <- uniform_filter(reshape(values, rows, cols), SMOOTH_WINDOW)
    return ElevationGrid(surface, bounds, cell_size_m,
                         extrapolated_fraction = mean(outside),
                         vertical_resolution_m = contours.interval)
```

Why square cells matter

The D8 model compares drops to eight neighbours over distances of 1 and $\sqrt{2}$ cell widths. If cells were rectangular those distances would be wrong and flow would bias along one axis. Solving for the shorter side rather than fixing both is what keeps the model valid for maps of any aspect ratio.

3.3 Stage 3 — Depression filling

Routing on a raw interpolated surface strands water in every hollow, and flow accumulation becomes meaningless. Priority-flood filling (Wang & Liu, 2006) raises each pit to its spill level. The fill depth added at a cell is not a by-product to discard — it *is* the depth of the hollow sitting there, which Stage 6 uses directly as evidence of natural storage.

Algorithm 3 — Priority-flood sink fill

```
function fill_sinks(z, epsilon = 0):
    filled <- copy of z
    queue  <- min-heap seeded with ALL BOUNDARY cells, keyed by elevation
    closed <- boundary cells marked

    while queue not empty:
        (level, r, c) <- pop lowest from queue
        foreach (nr, nc) in 8-neighbours of (r, c):
            if inside grid and not closed[nr, nc] then
                closed[nr, nc] <- true
                // A cell can be no lower than the lowest rim reaching it
                filled[nr, nc] <- max(filled[nr, nc], level + epsilon)
                push (filled[nr, nc], nr, nc) onto queue
    return filled

function depression_depth(z):
    return max(0, fill_sinks(z, 0) - z)     // head of water standing per cell

function conditioned_surface(z):
    return fill_sinks(z, epsilon = 1e-4)    // tilt flats so flow is defined
```

The ϵ tilt matters. A filled pit is a dead-flat lake in which no cell has a downhill neighbour, so flow direction is undefined across it. Adding 10^{-4} m per step of the flood restores a drainage path without changing any elevation meaningfully (Barnes et al., 2014).

3.4 Stage 4 — D8 flow routing

Algorithm 4 — Flow direction and accumulation

```
function flow_direction(z, cell_size_m):
    // D8: each cell drains to its steepest downhill neighbour of eight.
    padded <- z padded by one ring of +infinity   // off-grid can never win
    best   <- zeros;  direction <- filled with -1 // -1 marks a pit
```

```
    foreach k, (dr, dc) in the 8 offsets:
        neighbour <- padded shifted by (dr, dc)
        slope     <- (z - neighbour) / (cell_size_m * dist[k])
        steeper   <- slope > best
        best      <- where(steeper, slope, best)
        direction <- where(steeper, k, direction)
    return direction

function flow_accumulation(z, direction):
    // Water only moves downhill, so processing cells from HIGHEST to LOWEST
    // guarantees a cell's total is final before it is passed on.
    accumulation <- ones(size of z)              // each cell counts itself
    foreach cell in cells sorted by z descending:
        d <- downstream neighbour of cell via direction
        if d exists then
            accumulation[d] <- accumulation[d] + accumulation[cell]
    return accumulation
```

Sorting by elevation replaces the usual recursive traversal with a single linear pass, which is what keeps a 131×160 grid inside half a second.

3.5 Stage 5 — What the ground already is

Before any site is proposed, the map is split into ground a farm pond may occupy and ground it may not. The split is by *upstream area in hectares*, not by any property of this particular map, so it transfers unchanged to other terrain.

Class	Test	Consequence
Farm pond ground	upstream ≤ 25 ha	pond proposed
Nala	$25 <$ upstream ≤ 100 ha	bund / percolation tank
River	upstream > 100 ha	no structure
Floodplain	within 40 m buffer, HAND < 3 m	no structure
Still water	flat pooled area ≥ 2 ha	no structure

Table 3: Classification thresholds. All are physical, in hectares or metres.

The reasoning behind 25 ha

ICAR dryland manuals and the MGNREGA works schedule put a farm pond's catchment at roughly 1–10 ha. Past about 25 ha an ordinary storm delivers more than the pond can retain, and the engineering problem becomes a spillway rather than an embankment — a different structure, correctly reported as such rather than silently mis-recommended.

3.6 Stage 6 — Pond siting

Every eligible cell is scored on four things a contour map can genuinely answer. Each layer is normalised against the range present *in the uploaded map*, so the score is relative to that terrain and never to an absolute constant.

$$n(x) = \frac{x - \min x}{\max x - \min x} \tag{1}$$

$$L_{\text{depression}} = n(\text{fill depth}) \tag{2}$$

$$L_{\text{catchment}} = n(\log(1 + A)) \tag{3}$$

$$L_{\text{slope}} = n\left(\frac{1}{1 + \theta/\theta_{1/2}}\right), \quad \theta_{1/2} = 8^\circ \tag{4}$$

$$L_{\text{elevation}} = n(-z) \tag{5}$$

$$S = 100 \cdot \frac{\sum_i w_i L_i}{\sum_i w_i} \tag{6}$$

with weights $w = \{\text{depression } 0.30, \text{ catchment } 0.30, \text{ slope } 0.25, \text{ elevation } 0.15\}$. The log on accumulation matters: raw upstream counts span orders of magnitude, and without it a single trunk channel would saturate the layer and flatten every genuine difference elsewhere.

Algorithm 5 — Score, filter and rank candidate sites

```
function select_sites(grid, accumulation, depths, direction, max_sites):
    layers <- { depression: n(depths),
                catchment:  n(log1p(accumulation)),
                slope:      n(1 / (1 + slope_deg / 8)),
                elevation:  n(-grid.z) }
    score  <- 100 * weighted_sum(layers, WEIGHTS) / sum(WEIGHTS)

    // Rule out ground that cannot take a pond
    blocked <- river or floodplain or still_water or edge_ring
    // A busy channel with no measurable hollow is interpolation, not storage
    holds   <- max(0.30, grid.vertical_resolution_m)
    blocked <- blocked or (accumulation >= percentile(accumulation, 95)
                           and depths < holds)

    ranked <- cells not blocked, sorted by score descending
    sites  <- new list
    foreach cell in ranked:
        if count(sites) = max_sites then break
        if not spaced_apart(sites, cell, separation) then continue

        storage <- natural_storage(grid, depths, cell)
        inflow  <- upstream_cells * cell_area * 0.4 * 50mm / 1000  // one storm
        if inflow > storage.volume then
            reclassify as channel structure (bund + waste weir)
        else
            append PondSite(cell, score, storage, reasons) to sites
    return sites
```

Storage is not assumed from a design depth. The sink fill already gives the spill elevation, so the pond is grown outwards from the site up to that real rim:

Algorithm 6 — Natural storage by flood fill to the spill rim

```
function natural_storage(grid, depths, cell):
    if depths[cell] <= 0 then return zero storage    // nothing to hold; must
    be dug
    spill <- grid.z[cell] + depths[cell]             // the real rim it overtops
```

```
    flooded <- {cell};  queue <- [cell];  head_sum <- 0
    while queue not empty:
        (r, c) <- pop front
        head_sum <- head_sum + (spill - grid.z[r, c])
        foreach 4-neighbour (nr, nc) not already flooded:
            if grid.z[nr, nc] < spill then
                add (nr, nc) to flooded;  push onto queue

    return { depth_m:        depths[cell],
             spill_m:        spill,
             surface_area:   count(flooded) * cell_area,
             volume_m3:      head_sum * cell_area }
```

3.7 Stage 7 — Catchment delineation

With the site chosen, its catchment is exactly the set of cells whose flow pointers lead to it — found by walking the D8 network *upstream* from the outlet.

Algorithm 7 — Delineate the catchment of an outlet

```
function delineate(grid, direction, outlet):
    contributors <- invert(direction)      // downstream pointer -> upstream
    list
    cells <- {outlet};  queue <- [outlet]
    while queue not empty:
        cell <- pop front
        foreach up in contributors[cell]:
            if up not in cells then
                add up to cells;  push up onto queue

    area_m2   <- count(cells) * grid.cell_area_m2
    polygon   <- union of cell squares, exterior ring simplified
    flow_path <- longest upstream chain from outlet, in metres
    gradient  <- (max_z - min_z) / flow_path
    return Catchment(cells, area_m2, polygon, flow_path, gradient, relief,
    slope)
```

3.8 Stage 8 — Quantifying yield

Two standard formulae close the analysis. The Rational method gives the water a catchment yields; Kirpich gives how quickly it arrives, which is what sizes a spillway.

$$V = P \cdot A \cdot C \qquad \text{runoff volume (m}^3\text{)} \tag{7}$$

$$t_c = 0.0195\, L^{0.77} S^{-0.385} \qquad \text{time of concentration (min)} \tag{8}$$

where P is rainfall depth (mm), A catchment area (m^2), C the runoff coefficient (default 0.4), L the longest flow path (m) and S the average gradient. When no rainfall is supplied the response reports the yield per millimetre, $A \cdot C/1000$, so the caller may apply local rainfall themselves.

4 Generalization: no hard-coded results

This is the requirement most easily failed by accident, so it is worth being explicit about what is derived and what is constant.

Quantity	Source	Sample value
Geographic extent	vertex min/max	21.240–21.264 N
Elevation range	contour z values	267–298 m
Contour interval	median level gap	1.0 m
Grid rows × cols	extent aspect ratio	131 × 160
Cell size	haversine / cols	20.6 m
Score normalisation	map's own min/max	per-upload
Pond location	argmax of score	computed
Catchment area	upstream cell count	computed

Table 4: Every map-specific quantity is derived at request time.

The only constants in the codebase are physical or engineering ones that hold anywhere: the 25 ha and 100 ha structure thresholds, the 8° slope half-score, the 50 mm storm test, Kirpich's coefficients, and the D8 neighbour geometry. None encodes a location.

A concrete check

Uploading a different contour map produces a different grid shape, a different cell size, different normalisation ranges and a different site — with no code change. The companion endpoint `/analyzeArea` exercises the same core on terrain downloaded for arbitrary world coordinates, which is the strongest available evidence that the analysis is not tuned to the sample.

5 API Documentation

5.1 Endpoints

Method	Path	Purpose
`POST`	`/api/analyzeContour`	Analyse an uploaded contour map
`POST`	`/api/findCatchment`	Alias of the above
`POST`	`/api/analyzeArea`	Analyse by coordinates (no file)
`GET`	`/api/elevation`	Point elevation lookup
`GET`	`/api/health`	Liveness check
`GET`	`/docs`	Interactive OpenAPI documentation

Table 5: Both prefixed (`/api/...`) and bare (`/...`) paths are served.

The analysis routes are POST-only

`/api/analyzeContour` carries a file in the request body and therefore cannot be opened from a browser address bar — a `GET` correctly returns `405 Method Not Allowed`. To exercise it interactively, open `http://10.1.75.53:5249/docs`, which provides an upload form bound to the live endpoint.

5.2 Request

```
Content-Type:  multipart/form-data
```

Field	Type	Default	Meaning
`file`	file	required	Contour map, `.kml` or `.kmz`, ≤ 64 MB
`resolution`	int	160	Cells along the longer side (48–260)
`max_sites`	int	5	Candidate sites to return
`runoff_coefficient`	float	0.4	C in the Rational method
`rainfall_mm`	float	null	If given, absolute runoff volume is computed

Table 6: Request parameters. Only `file` is mandatory.

5.3 Response schema

```
200 OK --- abridged
  {
    "success": true,
    "analysis_id": "ca_aff346d081",
    "generated_at": "2026-09-01T17:28:13+00:00",
    "source":   { "kind", "format", "contour_lines", "vertices",
                  "elevation_levels", "contour_interval_m", "bounds" },
    "terrain":  { "min_elevation_m", "max_elevation_m", "relief_m",
                  "mean_slope_deg", "mapped_area_km2", "grid" },
    "pond_site":{ "rank", "latitude", "longitude", "elevation_m",
                  "slope_deg", "depression_depth_m", "upstream_hectares",
                  "height_above_drainage_m", "structure", "score",
                  "rating", "score_breakdown", "storage", "reasons" },
    "catchment":{ "outlet", "area_m2", "area_km2", "area_hectares",
                  "cell_count", "perimeter_m", "relief_m",
                  "longest_flow_path_m", "average_gradient",
                  "time_of_concentration_min", "runoff" },
    "alternative_sites":  [ ... ],
    "channel_structures": [ ... ],
    "watercourses":       { ... },
    "overlays":           { "elevation", "hillshade", "watercourse" },
    "geojson":            { "type": "FeatureCollection", "features": [ ... ] },
    "warnings":           [ ... ]
  }
```

5.4 Error responses

Status	Meaning	Typical cause
400	Bad request	Not a KML/KMZ, or unreadable archive
405	Method not allowed	`GET` on a POST-only analysis route
413	Payload too large	Upload exceeds 64 MB
422	Unprocessable	Fewer than 3 contour lines or 2 distinct levels
500	Analysis failed	Interpolation produced an incomplete surface

Table 7: Errors share one shape: {`success, error, detail`}.

6 Demonstration

Run against the provided `contours_1m.kml`:

```
Request

  curl -X POST \apiroute \
       -F "file=@contours_1m.kml" \
       -F "resolution=160"
```

Observed: `HTTP 200`, 43,588 bytes, in approximately 0.5 s.

6.1 What was read from the file

Property	Derived value
Contour lines	1,355
Vertices	159,113
Distinct elevation levels	32
Contour interval	1.0 m
Elevation range	267.0 – 298.0 m
Bounds (S, W, N, E)	21.2398, 81.2814, 21.2636, 81.3126

Table 8: Stage 1 output — entirely derived from the upload.

6.2 Reconstructed terrain

Grid	131 × 160 cells
Cell size / area	20.605 m / 424.57 m^2
Mapped area	8.8989 km^2
Elevation (min/mean/max)	267.55 / 283.83 / 296.67 m
Relief	29.13 m
Mean / max slope	2.37° / 13.36°
Extrapolated fraction	0.0755

Table 9: Stage 2–3 output.

6.3 Recommended pond site and its catchment

Pond site (rank 1)	
Location	21.249651 N, 81.286503 E
Structure	Farm pond
Score / rating	54.3 — Moderate
Ground elevation	277.38 m
Slope	1.11°
Depression depth	2.74 m
Height above drainage	5.66 m
Spill elevation	280.12 m
Water surface area	21,228.3 m^2
Storage volume	**26,811.3 m^3**
Catchment	
Outlet	site coordinates
Area	**9.47 ha** (94,678.2 m^2)
Cell count	223
Perimeter	1,867.6 m
Elevation range	275.66 – 285.27 m (relief 9.61 m)
Mean / max slope	2.84° / 7.48°
Longest flow path	601.4 m
Average gradient	0.01599
Time of concentration	13.2 min
Runoff yield	37.87 m^3 per mm of rainfall
Share of mapped area	1.06 %

Table 10: Primary result for the sample contour map.

6.4 Arithmetic check

Both reported quantities reproduce from the formulae in §3, which is a useful guard against a plausible-looking but wrong number:

$$\text{yield} = \frac{A \cdot C}{1000} = \frac{94678.2 \times 0.4}{1000} = 37.87 \text{ m}^3/\text{mm} \quad \checkmark \tag{9}$$

$$t_c = 0.0195 \times 601.4^{0.77} \times 0.01599^{-0.385} = 13.2 \text{ min} \quad \checkmark \tag{10}$$

6.5 Ground the analysis refused

The map is not uniformly available for a farm pond, and the service says so rather than proposing a site in a river:

River	51.46 ha
Floodplain	229.18 ha
Nala	9.38 ha
Truncated channels	1.70 ha
Still water	0.00 ha
Excluded fraction	**32.59 %**

Table 11: Stage 5 output. Roughly a third of the map carries no pond proposal.

Where a pond is inappropriate the service proposes the structure that *is* appropriate rather than returning nothing. The best such alternative on this map is a nala bund at 21.247414 N,

81.289911 E, serving 51.46 ha, scoring 65.5 (*Good*) — a higher score than the pond, correctly reported as a different structure.

6.6 Map output

Alongside the JSON, the response carries GeoJSON (12 features: catchment polygon, site markers, stream network, longest flow path) and three PNG overlays — elevation, hillshade and watercourse classification — georeferenced to the analysis bounds.

> **Figure placeholder**
>
> Save the map screenshot as `report/figures/catchment-map.png` and recompile; the figure and its caption will appear here automatically.

7 Extensibility to Phase 2

The architecture anticipates generalization in four specific ways.

1. **Terrain source is pluggable.** `build_grid` and `grid_shape` are shared by the contour path and the DEM-download path. A grid built from a downloaded raster has exactly the geometry one built from contours would, so every downstream stage is source-agnostic. Adding GeoTIFF or Shapefile input means writing one reader that returns a `ContourSet` — nothing in hydrology, siting or the API changes.
2. **Thresholds are physical, not positional.** Structure classification is in hectares and metres. Moving to a different district, state or country requires no retuning.
3. **Resolution is a request parameter.** The same map can be analysed coarsely for a fast preview or finely for a design study (48–260 cells per side) without redeployment.
4. **The core is HTTP-free.** `core/` imports nothing from FastAPI. The identical pipeline can be driven from a batch job, a scheduled task or a notebook in Phase 2.

7.1 Known limitations

Stated plainly, since they bound what the current results mean:

- Interpolation between contours cannot recover features finer than the contour interval; on this map, hollows shallower than 1 m are not distinguishable from smoothing artefacts, which is why the channel test uses the interval as its floor.
- About 7.6 % of the sample grid lies outside the contour convex hull and is nearest-neighbour extrapolated; the outer ring is excluded from siting for this reason.
- Catchments are clipped by the map extent. Flow entering from beyond the mapped area is not counted, and the response emits a warning when a candidate's catchment touches the boundary.
- Runoff uses the Rational method, which is appropriate for small catchments (< 25 ha here) but not for basin-scale routing.

8 Reproducing the result

```
Local
```

```
git clone \repourlraw
cd catchment_analyzer
python3 -m venv .venv && .venv/bin/pip install -r backend/requirements.txt
.venv/bin/uvicorn backend.main:app --host 0.0.0.0 --port 5249

curl -X POST http://localhost:5249/api/analyzeContour -F "file=@contours_1m.kml"
```

```
Docker
  docker compose up --build      # API and UI published on port 5249
```

References

[1] O'Callaghan, J.F. & Mark, D.M. (1984). The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics and Image Processing*, 28(3), 323–344.

[2] Wang, L. & Liu, H. (2006). An efficient method for identifying and filling surface depressions in digital elevation models. *International Journal of Geographical Information Science*, 20(2), 193–213.

[3] Barnes, R., Lehman, C. & Mulla, D. (2014). Priority-flood: an optimal depression-filling and watershed-labeling algorithm for digital elevation models. *Computers & Geosciences*, 62, 117–127.

[4] Kirpich, Z.P. (1940). Time of concentration of small agricultural watersheds. *Civil Engineering*, 10(6), 362.

[5] Indian Council of Agricultural Research. *Dryland Agriculture Manual — Farm Pond Design.*